



Software Design Specification

for

Hospital Appointment Booking App

Prepared by Group 9

M.Ibrahim Khan Tareen

Mian Khubaib Subhani

Haris Tahir Rana

Samie Ahmad

Usman Shafi

Date 09-03-25

Effort 40 hrs

Instructor

Naveed Arshad

Sulaiman Shahid

SBASSE-LUMS

Contents

CONTENTS	21
INTRODUCTION	1
1.1 CHANGE LOG	1
1.2 DOCUMENT PURPOSE	1
1.3 Product Scope	
2 OVERALL DESCRIPTION	22.1
SYSTEM OVERVIEW	22.2
SYSTEM CONSTRAINTS	22.3
ARCHITECTURAL STRATEGIES	43
SYSTEM ARCHITECTURE	53.1
SYSTEM ARCHITECTURE	4
3.2 DATABASE MODEL	5
3.3 EXTERNAL INTERFACE REQUIREMENTS	12
4 USE CASE VIEW	6
4.1 USE CASE TABLE	6
4.2 USE CASE DIAGRAM	6
4.3 Use Case Description	
5	USER INTERFACE DESIGN
	225.1
	DESCRIPTION OF THE USER INTERFACE
	225.2
	DATA FLOW DIAGRAMS
	225.3
	SCREENS
	235.4
	USER INTERFACE DESIGN RULES
	34 APPENDIX A - GROUP LOG
	35 APPENDIX B – REFERENCES
	36
A	Definitions, Acronyms, and Abbreviations
B	References and Acknowledgements

1 Introduction

1.1 Change log

1.2 Document Purpose

The purpose of this Software Development Specification (SDS) document is to provide a comprehensive and detailed description of the Hospital Appointment Booking Application, a digital healthcare solution designed to streamline patient-doctor interactions and improve healthcare management. This document serves as a blueprint for the development team, and other relevant parties, outlining the system's requirements, architecture, design, and functionality. It ensures a clear understanding of the project's objectives, scope, and constraints, guiding the development process from conception to deployment.

This SDS document also acts as a reference for evaluating the system's compliance with specified requirements, ensuring that the final product meets the needs of its users—patients, and doctors—while adhering to performance, security, and usability standards.

1.3 Product Scope

The Hospital Appointment Booking Application is a standalone software product designed to address inefficiencies in traditional healthcare management systems. It provides a user-friendly platform for patients to book, reschedule, and cancel appointments while enabling doctors to manage their schedules and communicate with patients in real time. The application integrates modern technologies such as AI chatbots, cloud-based databases, and push notifications to deliver a seamless and secure user experience.

This SDS document covers the entire scope of the application, including its core features such as user registration, appointment scheduling, AI-driven doctor recommendations, in-app messaging, and role-based access control. It also describes the integration of external systems and services, such as third-party APIs for authentication, notifications, and AI functionality. While the document provides a holistic view of the system, it may also specify individual subsystems or modules, such as the reporting dashboard, to ensure clarity and granularity in the development process.

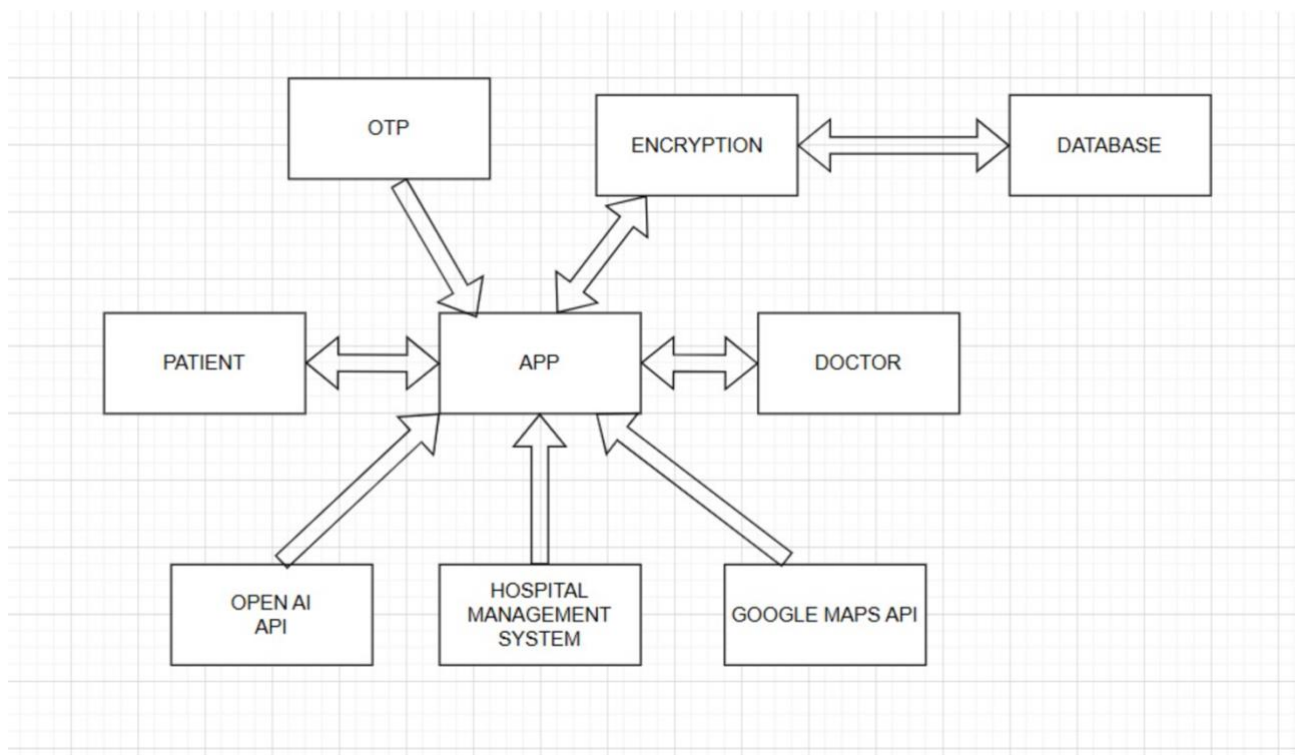
The application is designed exclusively for Android devices, with support for offline functionality and compatibility across a range of hardware configurations. By addressing the limitations of traditional appointment systems, this product aims to enhance healthcare accessibility, efficiency, and user satisfaction.

2 Overall Description

2.1 System Overview

The medical mobile application is designed to facilitate seamless communication between patients and doctors. It provides users with a comprehensive list of doctors, including their basic details such as qualifications, contact information, working hours, and affiliated hospitals or clinics. The application also maintains a secure record of patient's medical history, allowing users to store and retrieve their past checkups and frequently used medicines. Additionally, the application supports appointment booking and cancellation functionalities for both doctors and patients.

The system follows a client-server architecture where the mobile application (developed using React Native) serves as the client, interacting with a Python-based backend server. The backend handles business logic, authentication, and data encryption, ensuring a secure user experience. Patient and doctor data, including appointments and medical history, are stored in a MongoDB database. The system also integrates with external services such as Google Maps API and OpenAI API for chatbot applications.



2.2 System constraints

Several constraints impact the design and functionality of the medical mobile application:

- **Hardware and Software Environment:** The application is designed specifically for Android devices and requires a minimum of 4GB RAM, 5MB of storage, and Qualcomm Snapdragon 450 or equivalent. The minimum supported OS version is Android Marshmallow (6.0).
- **End-User Environment:** The intended users are both patients and doctors. No external devices are required for functionality, though future scalability may include integration with wearable health devices.
- **Availability or Volatility of Resources:** The system relies on third-party services such as Google Maps API for location services and an OpenAI API for a chatbot.
- **Standards Compliance:** The application complies with Pakistan's healthcare regulations and international standards followed in the US. Future deployments will adhere to additional regulations based on expansion to other regions.
- **Interface/Protocol Requirements:** The backend will support REST API communication, ensuring scalability and easy integration with third-party services.
- **Data Repository and Distribution Requirements:** All data is centrally stored in a MongoDB database for semi-structured data management and security.
- **Security Requirements:** Data encryption is enforced before storage, and user authentication is secured using OTP/email confirmation. Compliance with healthcare data security regulations ensures user data protection.
- **Memory and Other Capacity Limitations:** Each user is allocated up to 50MB of storage for medical history and appointment records.
- **Performance Requirements:** The expected response time for key operations (such as fetching doctor details or booking appointments) should be under 5 seconds to ensure a seamless user experience.
- **Network Communications:** The application optimizes bandwidth usage and uses WebSockets for real-time communication between doctors and patients.
- **Verification and Validation Requirements:** The system will be tested using unit testing, integration testing, and user acceptance testing (UAT) to ensure reliability and performance before deployment.
- **Other Means of Addressing Quality Goals:** A feedback mechanism will be integrated to allow users to report issues and suggest improvements.

- **Other Requirements:** The application will support the English language multilingual, with potential expansion to other languages in the future.

2.3 Architectural strategies

The design of the medical mobile application follows key architectural strategies to ensure efficiency, security, and scalability:

- **Technology Stack:** The frontend is developed using React Native, while the backend is implemented in Python. The database used for storage is MongoDB.
- **Security and Data Protection:** User authentication is enforced via OTP/email confirmation, and all sensitive medical data is encrypted before being stored in the database.
- **Data Storage and Management:** Medical history and appointment details are stored in MongoDB, ensuring structured data management with efficient retrieval mechanisms.
- **Scalability Considerations:** Although the system is initially designed for a small user base, the architecture allows for future scalability by optimizing database performance and expanding backend infrastructure.
- **User Interface and Experience:** The application follows modern UI/UX guidelines, ensuring a seamless experience for both patients and doctors.
- **Distributed Data and Control:** The system follows a centralized architecture for now.
- **Concurrency and Synchronization:** WebSockets are used for real-time communication, ensuring smooth interaction between users and the system.
- **Memory Management Policies:** The app requires a minimum of 4GB RAM and uses efficient caching mechanisms for frequently accessed data to optimize performance.
- **Communication Mechanisms:** The application uses RESTful APIs for most interactions, with WebSockets facilitating real-time updates.
- **Future Enhancements:** Plans include integrating video consultation features and AI-based medical services to enhance patient-doctor interactions and automate diagnosis assistance.

3 System Architecture

3.1 System Architecture

The Hospital Appointment Booking App follows a **layered client-server architecture**, designed to ensure modularity, scalability, and security. The system is decomposed into the following major components:

1. Frontend (Client):

- **Technology:** React Native (cross-platform mobile app).
- **Responsibilities:**
 - Rendering the user interface (e.g., appointment booking screens, medical history views).
 - Handling user inputs (e.g., form submissions, button clicks).
 - Communicating with the backend via RESTful APIs.
- **Subcomponents:**
 - **Login/Registration:** Handles user authentication.
 - **Appointment Scheduler:** Manages appointment booking, rescheduling, and cancellation.
 - **Chatbot Interface:** Provides a UI for interacting with the AI chatbot.
 - **Messaging Interface:** Enables real-time communication between patients and doctors.

2. Backend (Server):

- **Technology:** Python (handles business logic, authentication, and data encryption).
- **Responsibilities:**
 - Processing user requests (e.g., appointment scheduling, AI chatbot queries).
 - Managing database interactions (e.g., storing/retrieving patient/doctor data).
 - Integrating with third-party APIs (e.g., Google Maps for location services, OpenAI for chatbot functionality).
- **Subcomponents:**
 - **Authentication Module:** Manages user login, registration, and session management.
 - **Appointment Manager:** Handles appointment-related logic (e.g., availability checks, notifications).
 - **Chatbot Service:** Processes user queries and interacts with the OpenAI API.
 - **Messaging Service:** Facilitates real-time communication using WebSockets.

3. Database:

- **Technology:** MongoDB (stores semi-structured data).
- **Responsibilities:**
 - Storing and retrieving user data (e.g., patient profiles, doctor profiles, appointment records, medical history).

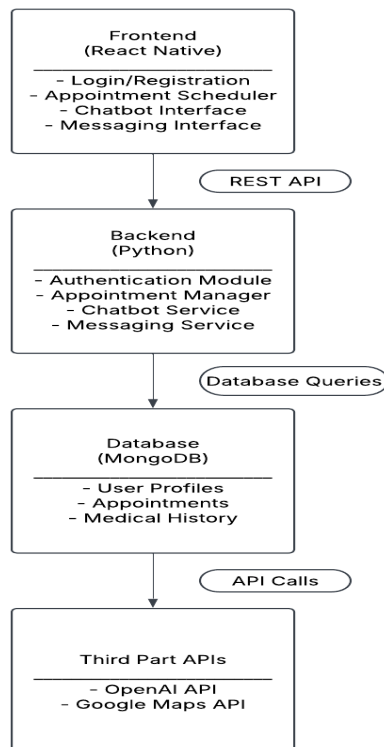
- Ensuring data security through encryption.
- **Subcomponents:**
 - **User Profiles:** Stores patient and doctor details.
 - **Appointments:** Manages appointment records.
 - **Medical History:** Stores patient medical records.
- 4. **Third-Party APIs:**
 - **Google Maps API:** Provides location-based services (e.g., finding nearby doctors).
 - **OpenAI API:** Powers the AI chatbot for symptom analysis and doctor recommendations.

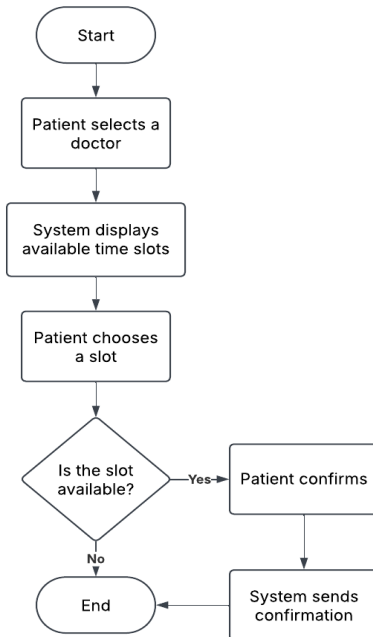
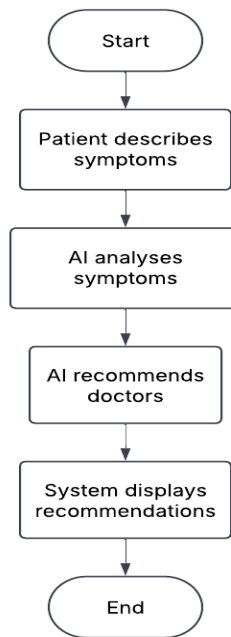
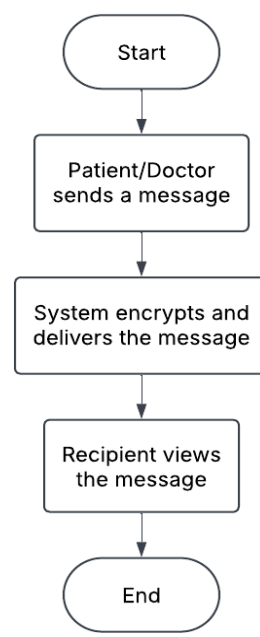
Rationale for Decomposition

The system was decomposed into these components to ensure:

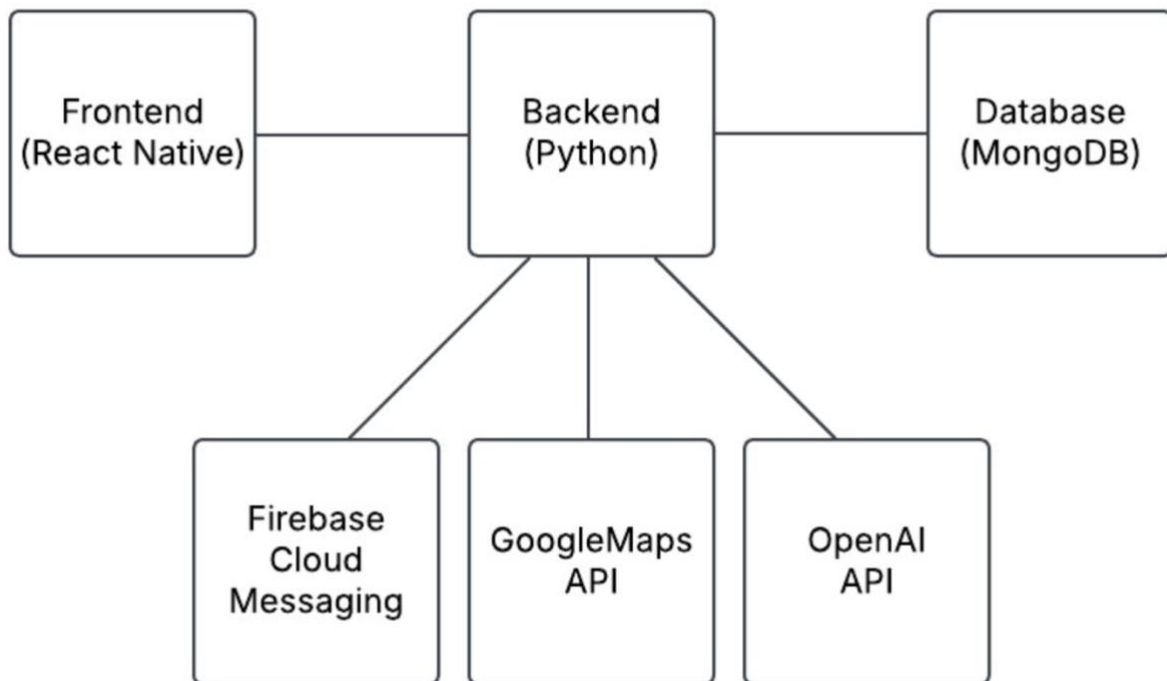
- **Modularity:** Each component has a clear responsibility, making the system easier to maintain and scale.
- **Scalability:** The client-server architecture allows for future enhancements (e.g., adding video consultation features).
- **Security:** Sensitive data is encrypted, and authentication is handled by the backend.

Component Diagram



Activity Diagram 1
(Appointment Scheduling)Activity Diagram 2
(AI chatbot)Activity Diagram 3
(Patient/Doctor chatbot)

High-Level Architecture (Box Line Diagram)



3.2 Database Model

Database Overview

The database is designed for a **Hospital Appointment Booking App**, enabling patients to book appointments with doctors, manage their medical history, and interact with hospitals. It also supports doctors in managing their schedules and hospitals in organizing their operations. The schema is normalized and uses MongoDB's flexible document model to store structured and semi-structured data.

Tables and Their Purpose

1. Hospitals

- Stores information about hospitals or clinics.
- Fields:
 - h_id: Unique identifier for the hospital.
 - name: Name of the hospital.
 - address: Physical address of the hospital.
 - number: Contact number of the hospital.
 - email: Email address of the hospital.
 - type: Type of healthcare facility (e.g., clinic, hospital).
 - opening_time and closing_time: Operating hours of the hospital.

2. Doctors

- Stores information about doctors working in hospitals.
- Fields:
 - d_id: Unique identifier for the doctor.
 - h_id: Reference to the hospital where the doctor works.
 - first_name and last_name: Doctor's name.
 - cnic: National identification number of the doctor.
 - address: Doctor's address.
 - gender: Gender of the doctor (male, female, other).
 - picture: URL or path to the doctor's profile picture.
 - education: Doctor's educational qualifications.
 - specialization: Medical specialty of the doctor (e.g., cardiology, dermatology).

3. Timeslots

- Stores available time slots for doctors at specific hospitals.
- Fields:
 - timeslot_id: Unique identifier for the time slot.
 - d_id: Reference to the doctor associated with the time slot.
 - h_id: Reference to the hospital where the time slot is available.
 - start_time and end_time: Start and end times of the time slot.
 - fee: Consultation fee for the time slot.

4. Patients

- Stores information about patients using the app.
- Fields:
 - p_id: Unique identifier for the patient.
 - first_name and last_name: Patient's name.
 - gender: Gender of the patient (male, female, other).
 - cnic: National identification number of the patient.
 - address: Patient's address.
 - blood_type: Patient's blood type.
 - emergency_contacts: A list of emergency contacts (stored as JSON objects).
 - medical_history: A list of medical records (stored as JSON objects).

5. Bookings

- Stores appointment bookings made by patients.
- Fields:
 - b_id: Unique identifier for the booking.
 - p_id: Reference to the patient who made the booking.
 - d_id: Reference to the doctor for the appointment.
 - h_id: Reference to the hospital where the appointment is scheduled.
 - timeslot_id: Reference to the time slot booked.
 - booking_start_time and booking_end_time: Start and end times of the booked appointment.
 - appointment_status: Status of the appointment (pending, confirmed, cancelled).

6. Authentication

- Stores user authentication details for both patients and doctors.
- Fields:
 - user_id: Unique identifier for the user (references p_id for patients or d_id for doctors).
 - number: Contact number for authentication.
 - email: Email address for authentication.

- password_hash: Hashed password for secure authentication.

Relationships Between Tables

1. **Doctor-Hospital Relationship:**

- A hospital (hospitals.h_id) can have many doctors, but each doctor (doctors.d_id) belongs to only one hospital.

2. **Booking-Hospital Relationship:**

- A hospital (hospitals.h_id) can have many bookings, but each booking (bookings.b_id) is associated with only one hospital.

3. **Booking-Doctor Relationship:**

- A doctor (doctors.d_id) can have many bookings, but each booking (bookings.b_id) is associated with only one doctor.

4. **Booking-Patient Relationship:**

- A patient (patients.p_id) can have many bookings, but each booking (bookings.b_id) is associated with only one patient.

5. **Booking-Timeslot Relationship:**

- Each booking (bookings.b_id) is linked to a specific time slot (timeslots.timeslot_id).

6. **Timeslot-Hospital Relationship:**

- Each time slot (timeslots.timeslot_id) is associated with a specific hospital (hospitals.h_id).

7. **Timeslot-Doctor Relationship:**

- Each time slot (timeslots.timeslot_id) is associated with a specific doctor (doctors.d_id).

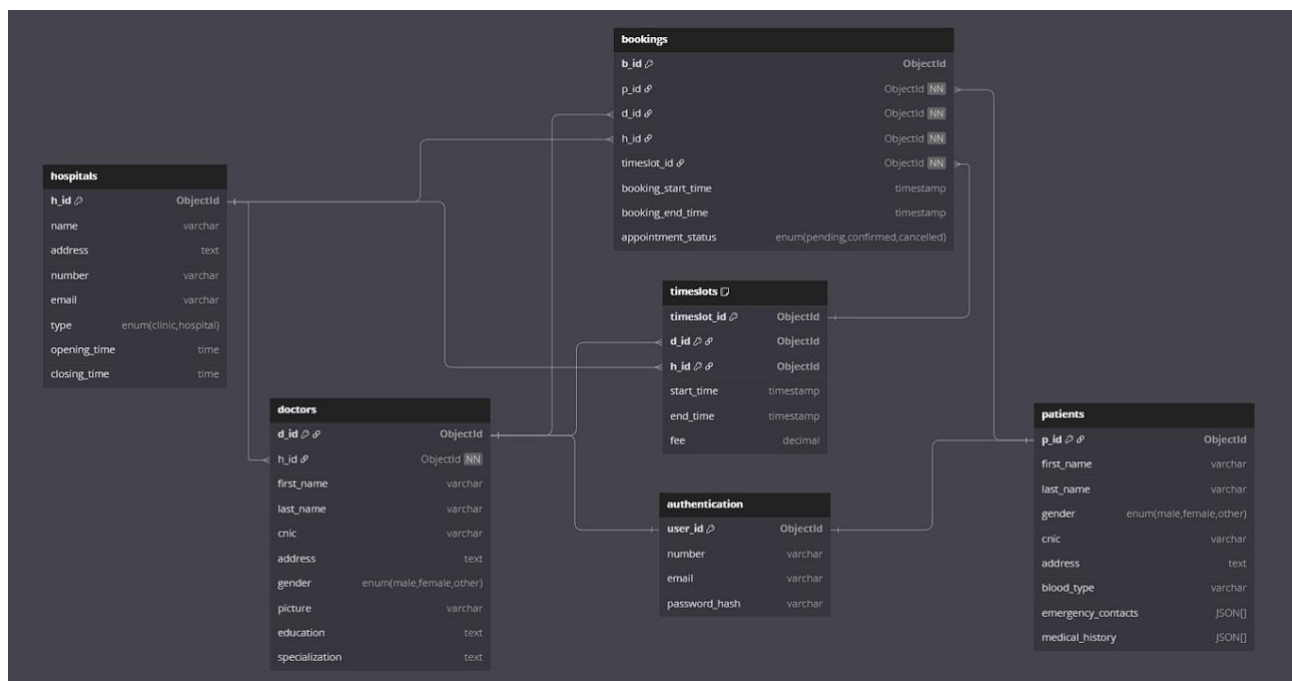
8. **Authentication-Patient Relationship:**

- A patient (patients.p_id) can have one authentication entry (authentication.user_id).

9. **Authentication-Doctor Relationship:**

- A doctor (doctors.d_id) can have one authentication entry (authentication.user_id).

3.2.1 Database scheme and detailed description



(courtesy: [Vertabelo](#))

3.2.2 Database

Choice of the Database – MongoDB

We have chosen MongoDB for our hospital appointment system due to its flexibility, scalability, and efficient handling of complex relationships. Below are the key reasons for this choice:

1. NoSQL Flexibility:

- MongoDB's schema-less design accommodates varying data structures, such as patient medical records and emergency contacts, without requiring rigid table structures. This simplifies future expansions.

2. Embedded Documents:

- Related data (e.g., emergency contacts, medical history) can be embedded within documents, reducing the need for joins and improving query performance.

3. Scalability & Performance:

- MongoDB supports horizontal scaling through sharding, ensuring the system remains responsive as the number of appointments grows. Indexing and caching further enhance search speeds.
- 4. **Efficient Relationships:**
 - One-to-many relationships (e.g., hospitals with multiple doctors, doctors with multiple time slots) are efficiently managed using references and embedded documents.
- 5. **JSON-like Query Language:**
 - MongoDB's BSON format (similar to JSON) integrates seamlessly with modern web applications, making queries intuitive and reducing data transformation overhead.
- 6. **Security:**
 - Role-based access control (RBAC) and encryption ensure the security of sensitive patient and booking data.

3.3 External Interface Requirements

3.3.1 User Interfaces

The Hospital Appointment Booking App provides a **Graphical User Interface (GUI)** designed for Android devices running Android 6.0 (Marshmallow) or later. The interface is optimized for ease of use, accessibility, and a seamless user experience, with distinct interfaces for patients and doctors.

1. **Logical Characteristics:**
 - **Role-Based Interfaces:**
 - **Patients:** Can search for doctors by specialty, hospital, and availability; book, reschedule, or cancel appointments; view medical history; and interact with an AI chatbot.
 - **Doctors:** Can manage schedules, update profiles, and communicate with patients.
 - **Navigation:** Intuitive menus and buttons guide users through the app's features.
 - **Real-Time Feedback:** Users receive immediate feedback (e.g., success messages, error notifications) for their actions.
2. **Optimization for Users:**
 - **Responsive Design:** The interface adapts to different screen sizes and orientations.
3. **Patient-Specific Features:**
 - **Search and Booking:** Patients can search for doctors by specialty, hospital, and availability, and book appointments through a calendar-style UI.

- **Medical Reports:** Patients can upload medical reports (PDFs, images) for doctors to review.
- **Communication:** Patients can chat with doctors for consultations or follow-ups.
- **Notifications:** Patients receive push notifications and email confirmations for appointments and reminders.

4. Doctor-Specific Features:

- **Schedule Management:** Doctors can set and modify available time slots.
- **Appointment Viewing:** Doctors can view booked appointments in a structured format.
- **Communication:** Secure messaging system for interacting with patients.

5. General Interface Characteristics:

- **Usability:** Simple and intuitive design with a focus on accessibility.
- **Language:** English-only interface in the initial release.
- **Authentication:** Email and password login (no biometric authentication in the initial release).

3.3.2 Hardware Interfaces

The app interacts with various hardware components of Android devices to deliver its functionality. Below are the key hardware interfaces:

1. Supported Devices:

- Compatible with Android 6.0 (Marshmallow) or later.
- Supports both smartphones and tablets (no iOS support in the initial release).

2. Data and Control Interactions:

- **Touchscreen Input:** Users interact with the app via touch gestures (e.g., taps, swipes).
- **Push Notifications:** Integrated with **Firebase Cloud Messaging (FCM)** for real-time push notifications (e.g., appointment reminders, chat messages).
- **Camera and Microphone:** Future updates may integrate video consultations, requiring access to the device's camera and microphone.

3. Document Uploads:

- Patients can upload medical reports using the device's camera or file picker.
- Basic document previewing is supported, but editing is not available.

4. Internet Dependency:

- The app requires an active internet connection for booking appointments, chat, and notifications.

5. Special Libraries:

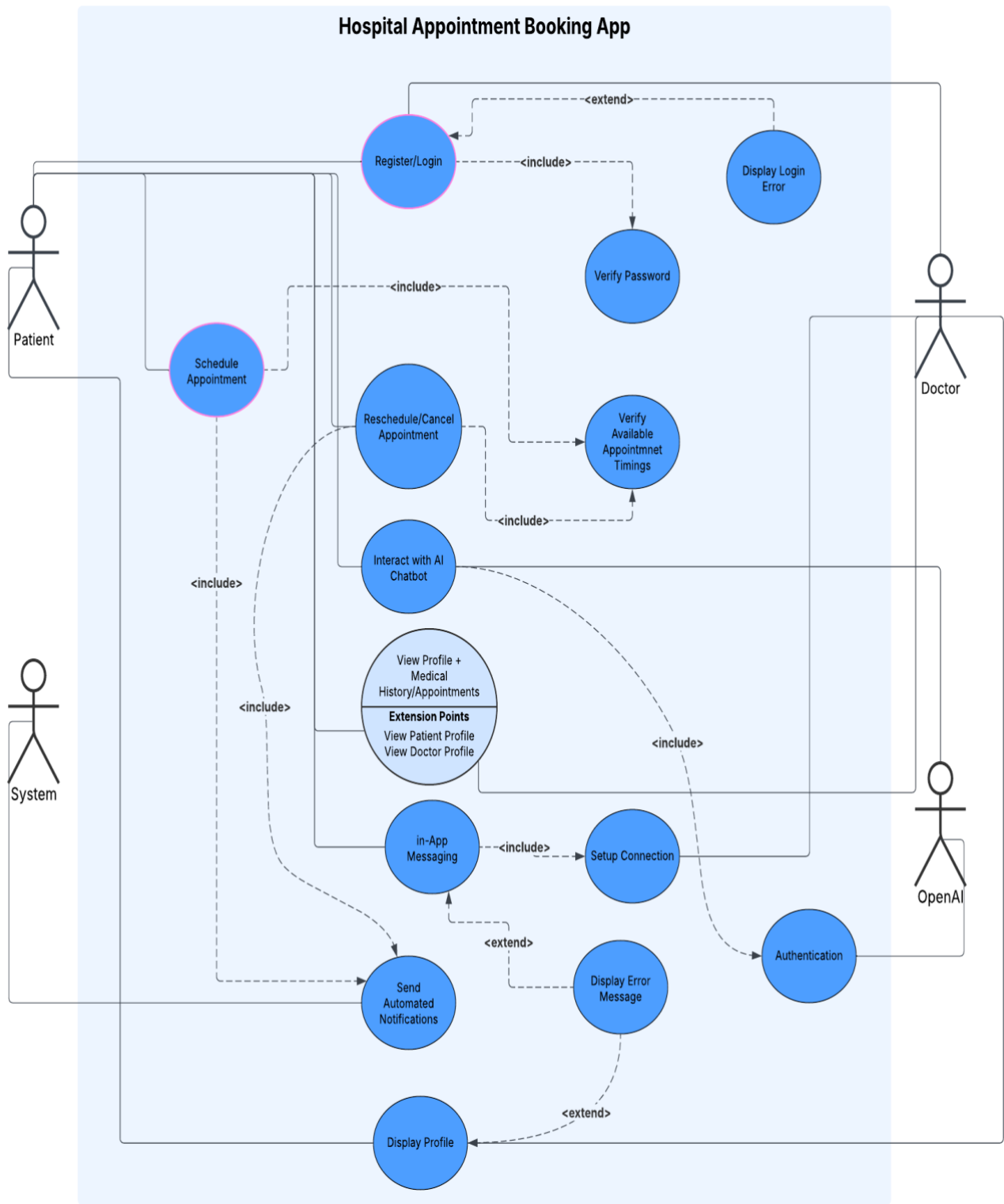
- **React Native:** Used for cross-platform mobile app development, ensuring compatibility with a wide range of Android devices.
- **Google Maps API:** Utilizes device GPS for location-based services (e.g., finding nearby hospitals).

4 Use Case View

4.1 Use Case Table

Primary Actors	Associated Use Cases
Patient	Register User
Patient	Login User
Patient	Schedule Appointment with Doctor
Patient	Reschedule appointment
Patient	Cancel Appointment
Patient	Interact with AI Chatbot
Patient	View Medical History
Patient	In-App Messaging
Patient	Search for Available Doctors
Doctor	Regsiter User
Doctor	Login User
Doctor	Update Dcotor Profile
Doctor	In App Messaging
System	Send Automated Notifications

4.2 Use Case Diagram



4.3 Use Case Description

Use Case 1: Register User

Actors:

- Patient
- Doctor

Preconditions:

- User does not have an account.

Basic Flow:

- User selects “Register.”
- User enters email, password, and role (patient/doctor).
- System validates inputs and creates an account.
- Confirmation email is sent.

Post Conditions: Account is created; user can log in.

Use Case 2: Login User

Actors:

- Patient
- Doctor

Preconditions:

- User has a registered account.

Basic Flow:

- User enters email and password.
- System authenticates credentials.
- User is redirected to their dashboard.

Post Conditions:

- User gains access to app features based on role.

Use Case 3: Schedule Appointment

Actors:

- Patient

Preconditions:

- Patient is logged in; doctor availability exists.

Basic Flow:

- Patient selects a doctor and date.
- System displays available time slots.
- Patient chooses a slot and confirms.
- System sends confirmation to patient and doctor.

Post Conditions:

- Appointment is booked; notifications sent.

Use Case 4: Reschedule Appointment

Actors:

- Patient

Preconditions:

- Appointment exists and is not past the deadline.

Basic Flow:

- Patient selects the appointment to reschedule.
- System shows available slots.
- Patient picks a new slot.
- System updates the appointment and notifies both parties.

Post Conditions:

- Appointment is rescheduled; notifications sent.

Use Case 5: Cancel Appointment

Actors:

- Patient

Preconditions:

- Appointment exists and is not past the deadline.

Basic Flow:

- Patient selects the appointment to cancel.
- Patient confirms cancellation.
- System removes the appointment and notifies both parties.

Post Conditions:

- Appointment is canceled; notifications sent.

Use Case 6: Update Doctor Profile

Actors:

- Doctor

Preconditions:

- Doctor is logged in.

Basic Flow:

- Doctor navigates to "Profile Management."
- Edits availability, working hours, or credentials.
- System updates the database.

Post Conditions:

- Updated profile is visible to patients.

Use Case 7: Interact with AI Chatbot

Actors:

- Patient, OpenAI

Preconditions:

- Patient is logged in.

Basic Flow:

- Patient describes symptoms in the chatbot.
- AI analyzes symptoms and recommends doctors.
- System displays recommendations.

Post Conditions:

- Patient receives doctor suggestions.

Use Case 8: Send Automated Notifications

Actors:

- System

Preconditions:

- Event triggers (e.g., appointment booked).

Basic Flow:

- System detects an event (e.g., new appointment).
- Generates email/push notification.
- Delivers notification to the user.

Post Conditions:

- Notification is sent successfully.

Use Case 9: In-App Messaging

Actors:

- Patient, Doctor

Preconditions:

- Both are logged in; an appointment exists.

Basic Flow:

- Patient/doctor sends a message.
- System encrypts and delivers it in real time.
- Recipient views the message.

Post Conditions:

- Message is securely delivered.

Use Case 10: View Medical History**Actors:**

- Patient

Preconditions:

- Patient is logged in; past appointments exist.

Basic Flow:

- Patient navigates to "Medical History."
- System retrieves records (appointments, prescriptions).
- Patient views details.

Post Conditions:

- Medical history is displayed.

Use Case 11: Access Analytics Dashboard**Actors:**

- Administrator

Preconditions:

- Admin is logged in.

Basic Flow:

- Admin opens the dashboard.
- System displays metrics (user activity, appointment trends).
- Admin analyzes data for decision-making.

Post Conditions:

- Dashboard data is accessible.

Use Case 12: Search for Available Doctors

Actors:

- Patient

Preconditions:

- Patient is logged in.

Basic Flow:

- Patient filters doctors by specialty, availability, or rating.
- System displays matching profiles.
- Patient selects a doctor to view details.

Post Conditions:

- List of available doctors is shown.

5 User Interface Design

note : *The User interface need to be complete in all functionality*

5.1 Description of the user interface

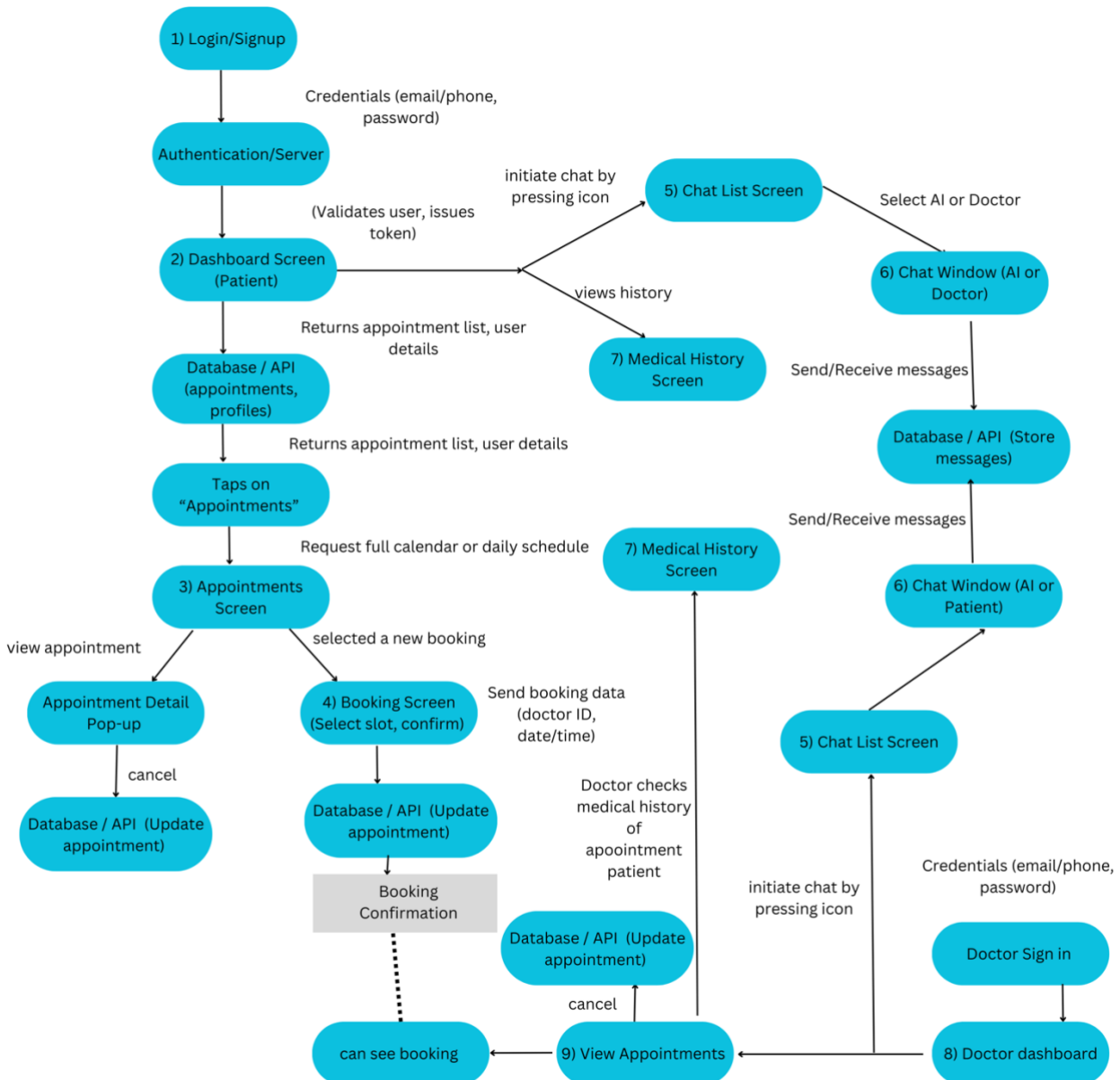
The user interface consists of a login/sign up setup, a patient dashboard and a doctor dashboard. The login page deals with logging and registering with email or google account, and is made easy to use. The patient dashboard shows the upcoming appointments, and has three sections; appointments, to view, make and cancel appointments, medical history, which shows the editable medical history, visible at the doctor's end, and doctors, which leads the user straight to the section of available doctors. The patient dashboard also features a chat icon, which leads the user to the chat service, where they can access the onboard first aid and general health AI assistance, or chat with doctors, regarding any issues after the visit. The doctor side dashboard features the appointments in that day, as well as an overview of the total appointments. The doctor can check all appointments, and can view the patient medical details, as well as having the ability to cancel them. The UI is designed to be user friendly for all age groups, and focusses on simplicity and straightforward, minimalistic design. It is intuitive, accessible and responsive

Rationale for using React Native:

React Native is an ideal choice for developing an Android doctor appointment management app because it offers cross-platform compatibility, near-native performance, and rapid development cycles. Its hot reloading feature and extensive library ecosystem speed up iteration and simplify integration of functionalities like scheduling, real-time notifications, and secure user authentication. While bridging to native code may be necessary for some features, well-documented approaches and a carefully curated dependency list ensure smooth collaboration between JavaScript and native modules. Overall, React Native's robust performance, large community support, and cost-effective code sharing make it a scalable and user-friendly solution.

Overall the UI is optimised for a wide range of users, as this app is supposed to be used by all demographics. It is easy to operate, fluid and looks clean and minimalistic. The colours and fonts are easy to perceive and read, and the mechanisms are straightforward and self-explanatory.

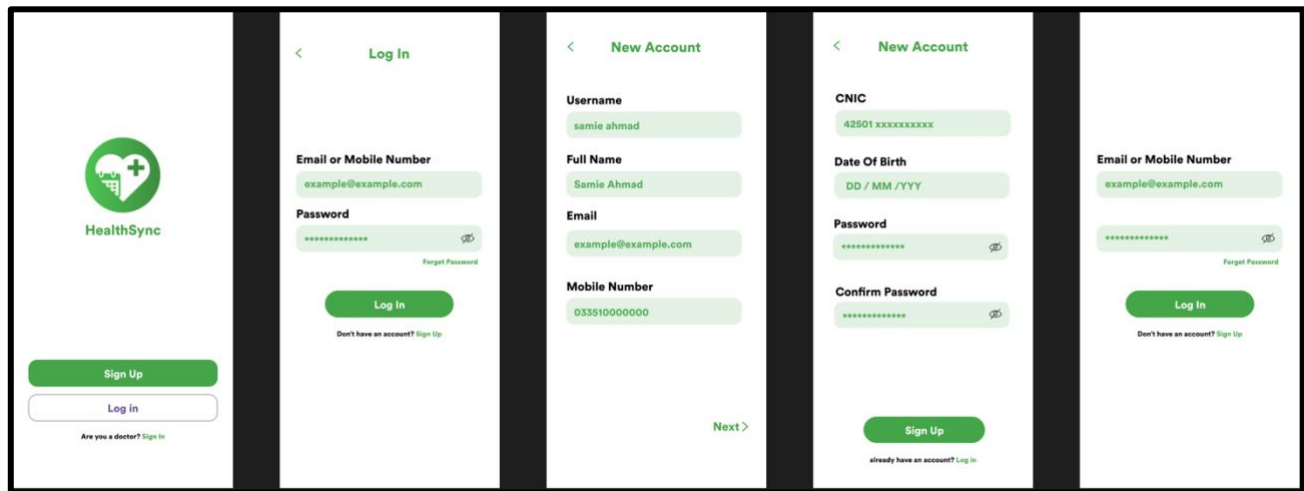
5.2 Data Flow Diagrams



5.3 Figma Screens (UI)

To do: Add Figma screen images with description – explain each item on the screen e.g. button, text field, etc.

Explain how the screen is mapped on one of the user requirements/use cases and Discuss the relationship between screens.



1. Login Screens:

Use Case: Case 1 and 2 Register and Login User

UI Design Elements

1. Clean Layout & Consistent Color Scheme
 - A predominantly green and white palette creates a calm, medical-themed feel and keeps the interface visually cohesive.
2. Minimalist Forms
 - Each screen focuses on a few form fields at a time (e.g., username, email, CNIC), reducing user overwhelm and making data entry straightforward.
3. Clear Calls to Action (CTA)
 - Prominent “Log In” and “Sign Up” buttons guide users clearly through the main actions.
 - “Forgot Password?” link is placed under the login fields for easy access.
4. Progressive Disclosure
 - The sign-up process is split into two screens (basic info then personal/secure details), which makes the form less intimidating and more user-friendly.

5. Readable Typography & Spacing

- Generous spacing around fields and legible text sizes ensure clarity and a professional appearance suitable for a healthcare context.

6. Doctor Login Button

- The doctor login is separated using a button, which accesses the doctor menus



2. Patient Dashboard:

Use Cases:

Use Case 3 (Schedule Appointment), 4 (Reschedule), 5 (Cancel): The Appointments button directly supports managing appointments.

Use Case 7 (Interact with AI Chatbot) & 9 (In-App Messaging): The chat icon at the top allows patients to either message their doctor directly or use the AI bot for first-aid/general health queries.

Use Case 10 (View Medical History): The “Medical History” tile provides quick access to past appointment records and prescriptions.

UI Elements Explanation

1. Upcoming Appointments Card (Carousel)

- Displays the next scheduled appointment (e.g., Doctor Zeeshan, date/time).
- Swiping or tapping would reveal multiple upcoming slots if more are booked.

2. Chat Icon

- Positioned in the top-right corner for easy access.
- Opens either direct messaging with doctors or an AI chatbot interface for first-aid and general health inquiries.

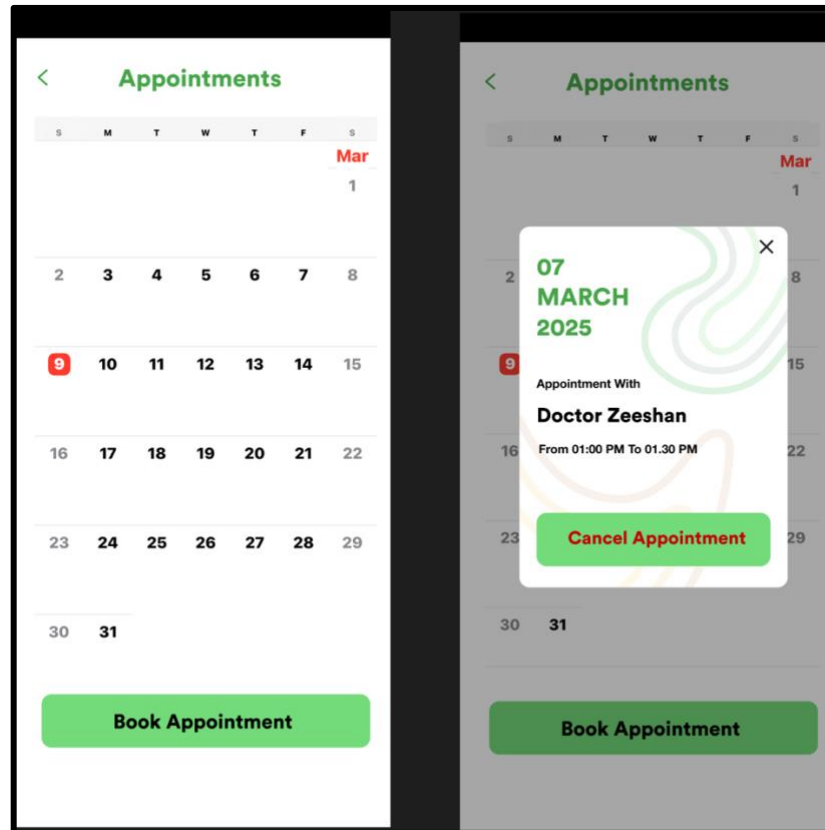
3. Three Main Buttons/Tiles

- **Appointments:** Central hub for scheduling, rescheduling, or canceling (supporting Use Cases 3–5).
- **Medical History:** Quickly view past appointments and related medical data (Use Case 10).
- **See Doctors:** Filter or browse available doctors by specialty, location, or rating (Use Case 12).

4. Profile Icon

- Typically located in the top-left or within a user menu.
- Allows login/logout, profile editing, and access to general settings

This home screen/dashboard layout gives patients a concise overview of their next appointment, direct access to their medical history, and quick ways to schedule or communicate—covering multiple key use cases in a single, user-friendly interface.



3. View Appointments:

Use Cases

- **Use Case 3 (Schedule Appointment):** The “Book Appointment” button initiates the process of selecting a doctor and time slot.
- **Use Case 5 (Cancel Appointment):** Tapping on an existing appointment displays its details in a pop-up, which includes a “Cancel Appointment” button to remove it.

UI Elements Explanation

1. Calendar View

- Shows all days of the month; users can tap a date to see any scheduled appointments or to choose a new slot.
- Provides a clear, at-a-glance overview of the user’s availability and upcoming commitments.

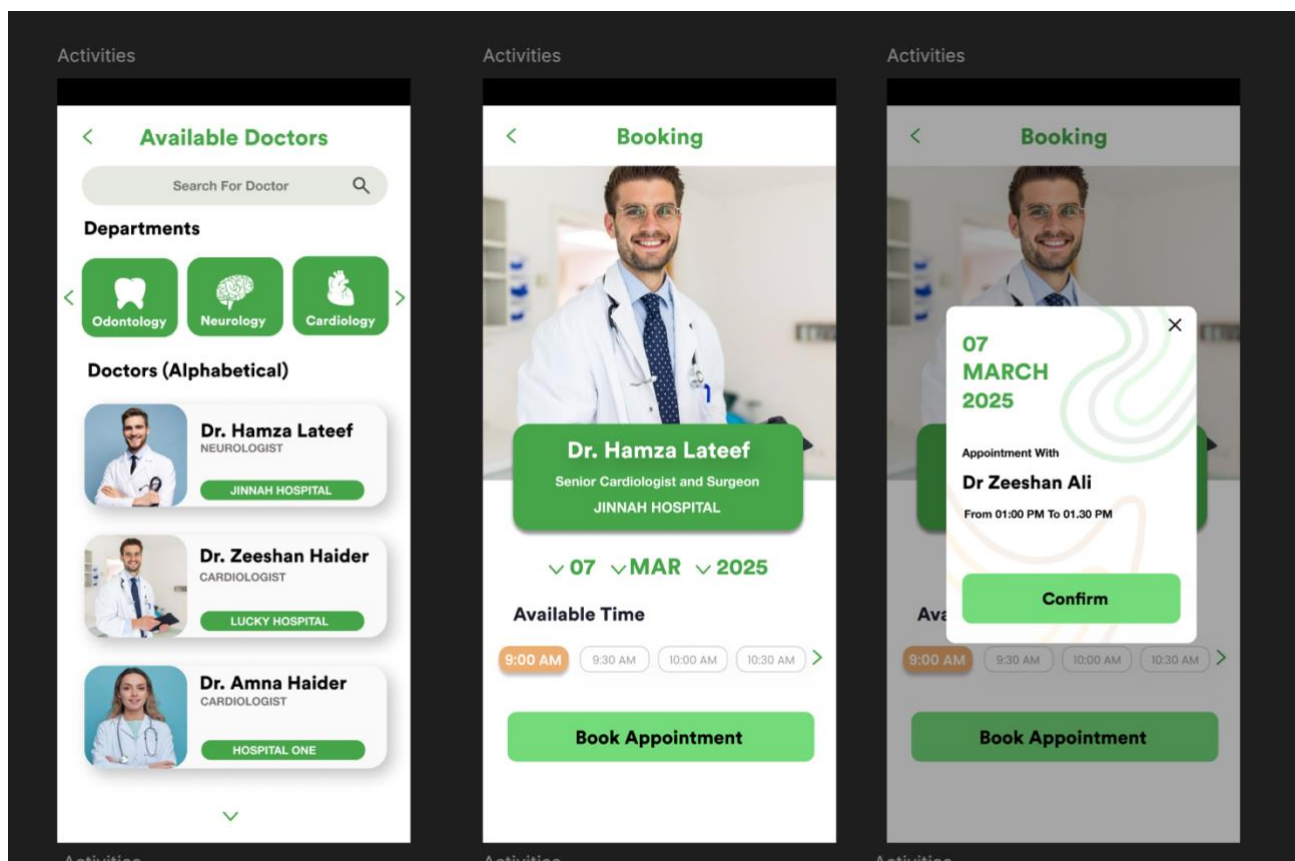
2. Pop-Up Appointment Details

- Displays the specific appointment (date, doctor's name, time).
- Contains the “Cancel Appointment” option, allowing quick cancellation without navigating to a separate screen.

3. “Book Appointment” Button

- Positioned at the bottom, it stands out as a primary call-to-action for scheduling new appointments.
- Clicking it typically leads to a flow where the user can select a doctor, date, and available time slot.

Overall, this design centralizes appointment management by combining both booking and cancellation features on a single, user-friendly screen.



4. Make Appointmemnts:

Use Cases

- **Use Case 12 (Search for Available Doctors):** The leftmost screen enables patients to filter doctors by specialty, view an alphabetical list, and use the search bar.
- **Use Case 3 (Schedule Appointment):** The middle and right screens guide patients through selecting a date/time slot, confirming the appointment, and then displaying the final booking

UI Elements

1. Doctor Categories & Search (Left Screen)

- **Categories:** Icons for departments (e.g., Cardiology, Neurology) to quickly filter relevant specialists.
- **Alphabetical List:** Scrollable list of doctors, making it easy to browse by name.
- **Search Bar:** Enables direct keyword searches for doctor names or specialties.

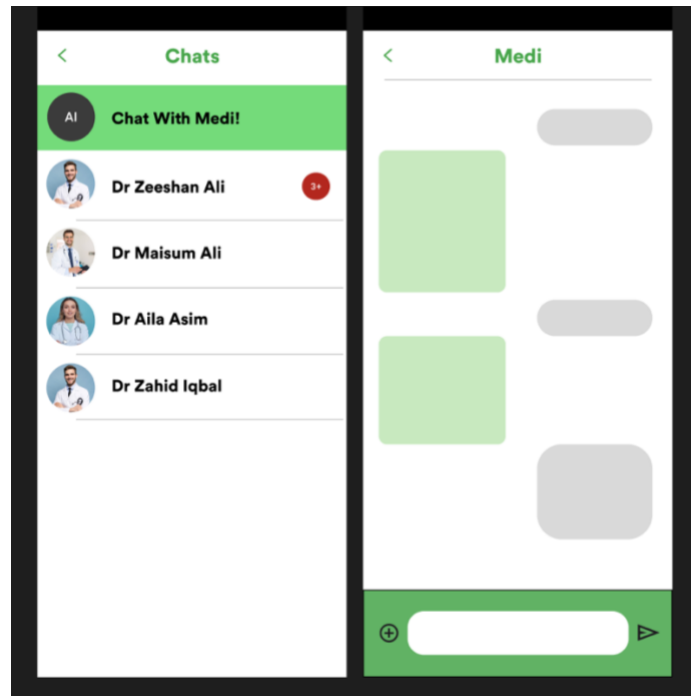
2. Booking Screen (Middle Screen)

- **Date Dropdown:** Allows patients to pick a desired date for the appointment.
- **Time Carousel:** Displays available time slots; unavailable slots are blacked out.
- **“Book Appointment” Button:** Primary action to proceed with scheduling once a slot is selected.

3. Confirmation Pop-Up (Right Screen)

- **Appointment Details:** Shows the selected doctor, date, and time.
- **Confirm Button:** Finalizes the booking, after which the appointment appears in the user’s “Appointments” tab.

This flow offers a straightforward process: search for a doctor, choose an available time, and confirm the booking—all in an intuitive, visually guided interface



5. Chat And AI Bot :

Use Cases

- **Use Case 7 (Interact with AI Chatbot):** The AI contact is pinned at the top, allowing users to quickly consult the chatbot for first-aid or general health queries.
- **Use Case 9 (In-App Messaging):** Users can chat directly with listed doctors to discuss appointments, follow-ups, or medical advice.

UI Elements

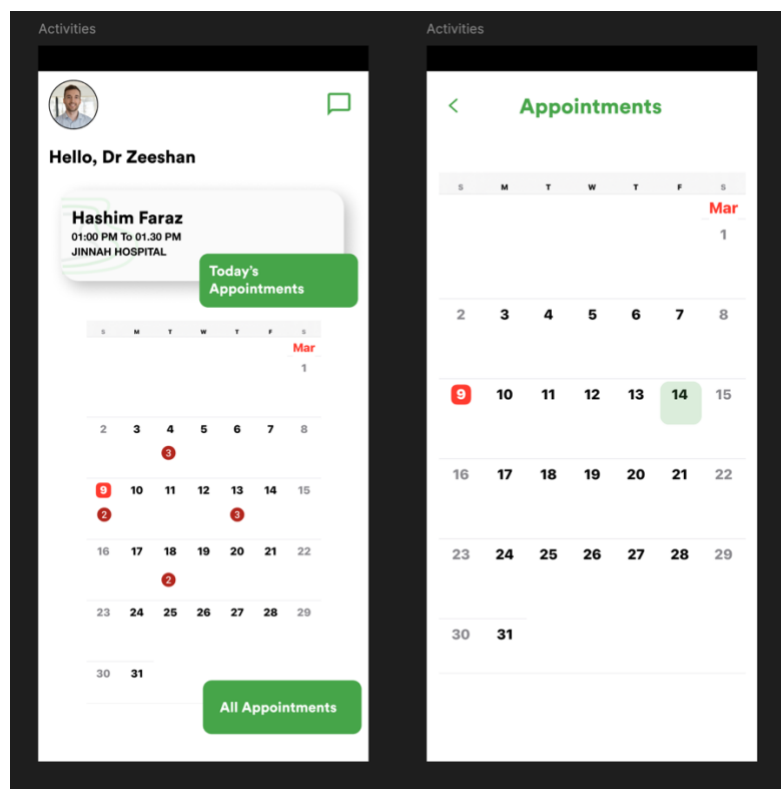
1. Chat List Screen

- **Pinned AI Chat:** Always appears at the top for easy access to automated health assistance.
- **Doctor Contacts:** Displays doctors' names and photos, each representing an ongoing conversation.
- **Unread Indicators (If any):** Could be shown (e.g., a badge) next to a doctor's name to highlight new messages.

2. Chat Window

- **Message Bubbles:** Show conversation flow between the user and the selected contact (doctor or AI).
- **Text Input Bar:** Positioned at the bottom, with a send button for quick message dispatch.
- **Header:** Indicates the current chat partner (e.g., “Medi” for AI, doctor’s name/photo for in-app messaging).

This setup provides a straightforward way to switch between AI-driven medical guidance and direct communication with healthcare professionals.



6. Doctor Dashboard :

Use Cases

While the original use cases focus largely on the patient’s perspective (scheduling, rescheduling, canceling), these screens address the **doctor’s viewpoint**—viewing and managing upcoming appointments. In practice, this ties into:

- **Use Case 3, 4, 5 (from the Doctor's Perspective):** Doctors see newly booked, rescheduled, or canceled appointments and can respond accordingly.

UI Elements & Flow

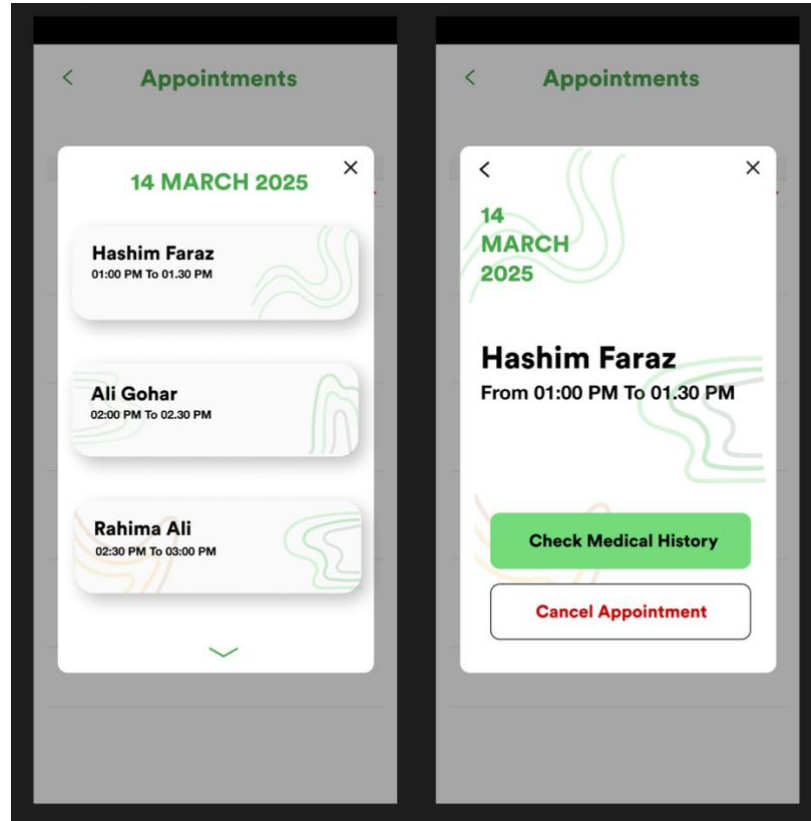
1. Doctor Dashboard

- **Daily Appointments Carousel:** Shows an at-a-glance view of the day's scheduled appointments.
- **Monthly Calendar:** Provides a broader overview; highlights days with bookings so the doctor can quickly see busy dates.
- **Chat:** Doctor end of the chat service, allowing them to communicate

2. All Appointments Tab

- **Detailed Day View:** Tapping on a specific date opens a list of that day's appointments.
- This lets doctors easily track or follow up on each appointment, accommodating any cancellations or reschedules.
-

By combining a quick daily carousel with a more detailed monthly calendar and "All Appointments" tab, the doctor's interface offers a streamlined way to monitor, confirm, or manage their schedule.



7. Doctor Appointments Screens :

Use Cases

- **Doctor's Perspective on Appointment Management:** While the original scheduling/canceling flows (Use Cases 3–5) are from the patient's viewpoint, these screens illustrate how doctors review and optionally cancel appointments.
- **Accessing Patient Details:** The “Check Medical History” button suggests an additional feature where the doctor can view a patient's records before or during an appointment (an extension of Use Case 10 for the doctor's side).

UI Elements & Flow

1. Day View of Appointments

- Tapping a date on the calendar reveals a scrollable list of that day's booked appointments.
- Each appointment card shows the patient's name and time slot.

2. Appointment Detail Pop-Up

- **Check Medical History:** Quick access to the patient's health records or past visit notes.
- **Cancel Appointment:** Allows the doctor to remove the booking if needed (e.g., emergency, scheduling conflict).
- **Date & Time Confirmation:** Confirms exactly when the appointment is scheduled.

This layout streamlines a doctor's workflow by consolidating all daily appointments into one list, providing instant access to patient records, and offering the option to cancel when necessary.

5.4 User interface design rules

Conventions & Standards Used in the UI Design

1. Consistent Color Palette

- A cohesive green-and-white scheme is applied across all screens to create brand consistency and reinforce a calm, health-oriented feel.

2. Platform Guidelines & Familiar Patterns

- Common mobile navigation patterns (e.g., top bars, bottom actions) are aligned with **Android Material Design** standards, ensuring intuitive navigation and predictable layouts.

3. Clear Typography & Readability

- Legible font sizes and sufficient line spacing help users quickly scan information—vital for medical data and time-sensitive actions.

4. Minimalistic Icons & Labels

- Simple, universally recognized icons (calendar, chat bubble, profile) paired with short labels reduce cognitive load and enhance discoverability.

5. Action-Oriented Buttons

- Primary actions (e.g., “Log In,” “Sign Up,” “Book Appointment”) use prominent, consistently styled buttons to guide users toward key tasks.

6. Progressive Disclosure

- Longer processes (e.g., account creation, booking flow) are broken into smaller steps or screens, preventing information overload and improving user focus.

7. User-Centric Terminology

- Labels, instructions, and notifications use plain language (e.g., “Your Appointments,” “Check Medical History”) to ensure clarity and inclusivity.

8. Responsive & Scrollable Layouts

- Lists and calendars are scrollable, adapting to varying device sizes and ensuring important content remains accessible without clutter.

9. Feedback & Confirmation

- Pop-up confirmations for critical actions (e.g., “Cancel Appointment,” “Confirm Booking”) provide clear feedback and prevent accidental changes.

10. Accessible Design Considerations

- Adequate contrast, consistent visual hierarchy, and navigable components aim to accommodate diverse user needs, aligning with accessibility best practices.

Appendix A - Group Log

Name	Contributions in this phase	Approx. Number of hours	Remarks
M.Ibrahim Khan Tareen	Firebase learning and Installation, Introduction	5	
Haris Tahir Rana	Use Case View part of SDS document, React Native Installation, Database design	8	
Mian Khubaib Subhani	Overall Description part of SDS document, Database design, MongoDB basics, React Native	8	
Usman Shafi	System Architecture part of SDS document, Database design, MongoDB basics	10	

Samie Ahmad	User Interface Design part of SDS document, Designed logos + UI design and Figma screens	10	

Appendix B – References

A Definitions, Acronyms and Abbreviations

<i>API</i>	A set of protocols and tools for building and interacting with software applications.
<i>BSON</i>	A binary-encoded serialization of JSON-like documents, commonly used by MongoDB.
<i>FCM</i>	Firebase Cloud Messaging, a service for sending push notifications and messages across platforms.
<i>GPS</i>	A satellite-based navigation system that provides accurate location and time information.
<i>GUI</i>	A Graphical User Interface that enables user interaction through visual icons and controls.
<i>iOS</i>	Apple's mobile operating system for iPhone, iPad, and iPod touch devices.
<i>JSON</i>	JavaScript Object Notation, a lightweight, human-readable data interchange format.
<i>MongoDB</i>	A document-oriented NoSQL database that stores data in flexible, JSON-like structures.
<i>NOSQL</i>	A class of database management systems that do not rely on fixed table schemas or SQL.
<i>OpenAI</i>	An AI research and deployment company focused on developing advanced artificial intelligence.
<i>OS</i>	An Operating System that manages computer hardware, software resources, and services for applications.
<i>OTP</i>	A One-Time Password that provides a single-use security code for authentication processes.
<i>RAM</i>	Random Access Memory, a type of computer memory that allows fast read and write access.
<i>RBAC</i>	Role-Based Access Control, a security model that restricts system access based on user roles.
<i>REST</i>	Representational State Transfer, an architectural style for designing stateless web services.
<i>UAT</i>	User Acceptance Testing, the phase where end users verify that a system meets their requirements.
<i>UI/UX</i>	User Interface/User Experience, disciplines focusing on the visual design and overall experience of digital products.

B References and Acknowledgments

- Zaeem TA Past SDS Document
- Figma: <https://www.figma.com/design/M3QIU4wEGgHsxndwRd7gBQ/SE-Pro?node-id=0-1&t=Zkbp7o2bC2f0XlP0-1>

Marking Rubric

Total marks: 45

System Constraints	3
System Architecture + UML	4
DB Model	5
Interface Reqs	3
Use Case View	10
Data Flow Diagrams	7
Figma Screens and UI Rules	8
References and Abbreviations	1
Concise and to-the-point descriptions	2
Writing Quality (Descriptions should be coherent and should covers all questions asked under the heading/follows format of document)	2